

— Beginner Exam Cheat Sheet

SPS1 Introduction · Networking, Sockets, T

1 · What Is a Socket?

A **socket** is the programming abstraction (API) that lets two processes communicate — either on the same machine (*inter-process communication*, *IPC*) or over a network (*inter-machine*). Think of it like a telephone jack: you plug in, dial, and talk.

Key Properties of Sockets

- Allow **both** inter-machine and inter-process communications.
- Created **without forking** — no need to spawn child processes.
- Provide a **simple API** for network I/O (send/recv, read/write).
- Increase **interoperability**: any language/OS that supports sockets can talk to any other.
- Networking service is provided by the **Operating System**.

■ Exam tip: A socket is basically a file descriptor. You can use `read()/write()` on it just like a file!

2 · Network Protocol Layers (Internet Stack)

Networks are complex. **Layering** organizes this complexity: each layer handles a specific job and exposes a clean interface to the layer above.

Layer	What It Does / Examples
Application	FTP, SMTP, HTTP — apps that use the network
Transport	TCP, UDP — process-to-process data transfer
Network	IP, routing protocols — routes packets from source to destination
Link	Ethernet, 802.11 (WiFi), PPP — transfer between neighboring devices
Physical	Bits on the wire (electrical signals, light pulses)

Why layering? Modularization means you can change one layer's implementation without touching others. E.g., switching from Ethernet to WiFi doesn't change how HTTP works.

■ Mnemonic (top → bottom): **All The Network Layers Please** → Application, Transport, Network, Link, Physical

3 · Encapsulation

Each layer **wraps** the payload from the layer above with its own header (and sometimes trailer). This is called **encapsulation**.

Layer adds →	Resulting PDU	Header Name
Application	Message (M)	(none)
Transport	Ht + M (segment)	Transport Header (Ht)
Network	Hn + Ht + M (datagram)	Network Header (Hn)
Link	Hi + Hn + Ht + M (frame)	Link Header (Hi)

At the receiver, each layer **strips** its header and passes the payload up. Routers only process up to the **Network** layer; switches up to the **Link** layer.

4 · IP — Internet Protocol

IP is a **datagram service** at the Network layer. It provides **best-effort** delivery — no guarantees of order, delivery, or uniqueness.

IP Key Concepts

- **Packet (datagram)**: data is split into chunks; each carries a destination IP address.
- **Independent routing**: packets of the same message can take different paths.
- **Best-effort**: IP can lose, reorder, or duplicate packets — it doesn't fix this.

IPv4 vs IPv6

Feature	IPv4	IPv6
Address size	4 bytes (32 bits)	16 bytes (128 bits)
Notation	Dotted decimal: 192.168.1.1	Hex groups: 2001:db8::1
Address space	~4.3 billion	340 undecillion
Compatibility	—	NOT compatible with IPv4
Auto-config	Manual or DHCP	Built-in auto-configuration

IPv4 Example Addresses

- **67.218.93.15** — a Google server (public address)
- **137.99.10.1** — a UConn gateway (public address)
- **Private ranges** (not routable on the public Internet):
 - 10.0.0.0 – 10.255.255.255
 - 172.16.0.0 – 172.31.255.255
 - 192.168.0.0 – 192.168.255.255

CIDR — Classless InterDomain Routing

Format: **a.b.c.d/x** — the x tells you how many bits are the **network (subnet) part**; the rest are the **host part**.

```
Example: 200.23.16.0/23
Binary:  11001000 00010111 0001000|0 00000000
          <--- 23 bits subnet ---><- host ->
All addresses 200.23.16.0 - 200.23.17.255 are in this subnet.
```

■ /24 = 256 addresses, /16 = 65536 addresses, /8 = 16M addresses. Smaller number after / → more hosts in the subnet.

5 · DNS — Domain Name System

DNS is the **phonebook of the Internet**. It translates human-readable domain names into IP addresses.

Domain name	An identification string (e.g., www.uconn.edu)
FQDN	Fully Qualified Domain Name — complete address for a specific host on the Internet
DNS server	Translates (resolves) a domain name → IP address
Hierarchy	Names are hierarchical: com → google → www
Example	www.google.com → 172.217.7.228

6 · Transport Layer: TCP vs UDP

Both TCP and UDP sit **above IP** and add process-to-process addressing via **ports**.

Feature	TCP	UDP
Connection	Connection-oriented (like a phone call)	Connectionless (like mailing a postcard)
Reliability	Reliable — guarantees delivery and order	Unreliable — packets may be lost or reordered
Flow control	Yes — prevents sender from overwhelming receiver	No
Congestion	Yes — backs off on network congestion	No
Direction	Bi-directional	Uni or bi-directional
Overhead	Higher (headers, handshake)	Lower (minimal headers)
Use cases	Web, email, file transfer	Video streaming, DNS, gaming
Socket type	SOCK_STREAM	SOCK_DGRAM

■ With UDP, packets can arrive out of order, be duplicated, or be lost entirely. Retransmission must be handled in your application code if needed.

7 · Ports

A server can offer **many services simultaneously**. A **port** is a 16-bit number (0–65535) that identifies which service/application a packet is destined for. Both TCP and UDP use ports.

Port	Service / Protocol
21	FTP (File Transfer Protocol)
22	SSH (Secure Shell)
25	SMTP (email sending)
80	HTTP (web traffic)
137–139	NetBIOS
443	HTTPS (encrypted web)

■ Ports 0–1023 are 'well-known' (reserved). Ports 1024–49151 are registered. Ports 49152–65535 are ephemeral/dynamic (used for client-side connections).

8 · Sockets API in C

A socket is the **programmatic abstraction to access ports**. Sockets work like files: you open them, read/write data, and close them.

socket() — Create a Socket

```
#include <sys/types.h>
#include <sys/socket.h>
int socket(int domain, int type, int protocol);
// Returns: file descriptor (>= 0) on success, -1 on error
// Close with: close(fd);
```

domain	AF_INET (IPv4) AF_INET6 (IPv6)
type	SOCK_STREAM (TCP) SOCK_DGRAM (UDP)
protocol	Usually 0 — OS picks the right protocol for domain+type
return	A file descriptor integer; -1 means error

```
// Example: create a TCP/IPv4 socket
int sfd = socket(AF_INET, SOCK_STREAM, 0);
if (sfd < 0) { perror("socket"); exit(1); }
```

send() — Send Data over TCP

```
#include <sys/types.h>
#include <sys/socket.h>
ssize_t send(int sfd, const void *buf, size_t len, int flags);
// Returns: bytes actually sent (may be < len!), or -1 on error
```

- **sfd**: the socket file descriptor returned by socket()
- **buf**: pointer to the data you want to send

- **len**: number of bytes you want to send
- **flags**: usually 0
- Return value can be **less than len** — that's why you need a loop!

Complete send() Loop Pattern

```
// Reliably send exactly n bytes from buf
sent = 0;
while (sent < n) {
    int r = send(sfd, buf + sent, n - sent, 0);
    if (r < 0) report_error(); // handle error
    sent += r;                // advance pointer
}
```

■ Never assume send() sends all bytes at once. Always use a loop that advances the buffer pointer by the actual bytes sent.

recv() — Receive Data over TCP

```
ssize_t recv(int sfd, void *buf, size_t len, int flags);
// Returns: bytes actually received (may be < len!), or -1 on error
// Returns 0 when the connection is closed by the other side
```

Complete recv() Loop Pattern

```
// Reliably receive exactly n bytes into buf
received = 0;
while (received < n) {
    int r = recv(sfd, buf + received, n - received, 0);
    if (r < 0) report_error(); // error
    if (r == 0) break;        // connection closed
    received += r;            // advance pointer
}
```

■ send() and recv() are the TCP equivalents of write() and read(). You can use read()/write() directly if you don't need any flags.

9 · Associating a Socket with an Address

Before a socket can communicate, it must be associated with:

- **An Internet address** (IPv4 or IPv6)
- **A port number**

Server sockets use **bind()** to attach to a port so clients can find them. Client sockets use **connect()** to reach a server.

Typical TCP Client Flow

1. `socket()` → create socket fd
2. `connect()` → connect to server IP + port
3. `send()` → send data
4. `recv()` → receive data
5. `close()` → close socket

Typical TCP Server Flow

1. `socket()` → create socket fd
2. `bind()` → attach to a local port
3. `listen()` → mark socket as passive (waiting for connections)
4. `accept()` → block until a client connects; returns new fd
5. `recv()` → receive data from client
6. `send()` → send response
7. `close()` → close connection fd (and server fd when done)

10 · Useful Shell Commands

```
# Scan open ports on localhost
nmap localhost

# List file descriptors for Internet sockets (add sudo for system-wide)
lsof -i

# Show networking subsystem info (all listening sockets)
netstat -l

# Show only IPv4 listening sockets
netstat -l -4
```

11 · Quick Reference Card

Term / Function	One-Line Definition
Socket	Endpoint for network communication; works like a file descriptor
IP	Network layer; best-effort packet delivery with addressing
TCP	Reliable, ordered, connection-oriented transport (SOCK_STREAM)
UDP	Unreliable, fast, connectionless transport (SOCK_DGRAM)
Port	16-bit number identifying a service on a host (0–65535)
DNS	Translates domain names to IP addresses
FQDN	Full domain name for a specific host, e.g. <code>www.uconn.edu</code>
CIDR	<code>a.b.c.d/x</code> notation; <code>x</code> bits = network part, rest = host part
Encapsulation	Each layer adds a header; receiver strips it going up

AF_INET	Address family for IPv4 sockets
AF_INET6	Address family for IPv6 sockets
SOCK_STREAM	Stream socket → TCP
SOCK_DGRAM	Datagram socket → UDP
socket(d,t,p)	Create socket; returns fd or -1 on error
send(fd,buf,n,0)	Send up to n bytes; loop until all sent
recv(fd,buf,n,0)	Receive up to n bytes; loop until all received
close(fd)	Close and release the socket
bind()	Attach socket to local address/port (server side)
listen()	Mark socket as passive; queue incoming connections
accept()	Block until client connects; returns new fd for that connection
connect()	Client connects to server address/port

12 · Common Exam Mistakes to Avoid

■■ Assuming **send()** sends everything: `send()` can return fewer bytes than requested. ALWAYS loop until all bytes are sent.

■■ Assuming **recv()** fills the buffer: `recv()` can return fewer bytes than `len`. ALWAYS loop until you've received everything you need.

■■ Confusing **TCP and UDP socket types**: TCP = `SOCK_STREAM`. UDP = `SOCK_DGRAM`. Know this cold.

■■ Forgetting to check return values: `socket()`, `send()`, `recv()` all return -1 on error. Always check!

■■ Mixing up **IP's job and TCP's job**: IP handles addressing and routing. TCP adds reliability, ordering, and flow control on top.

■■ Thinking **domain names = IP addresses**: They are not the same. DNS servers translate names → addresses.

■■ Assuming a socket is **UDP by default**: You must explicitly specify `SOCK_DGRAM` for UDP. The default example in class uses `SOCK_STREAM` (TCP).

■■ Forgetting that **protocol parameter is 0**: In `socket()`, always pass 0 for protocol — the OS picks the correct one.